

Mortgage Engine Implementation Walkthrough

How the Go port sizes a loan: the field-presence `calc` dispatch, the annuity math, the secant APR solver, the two-variable Newton crossover comparison, and the what-if row generator.

Scope. The Go mortgage package `internal/finance/mortgage/` (`mortgage.go`, `advisories.go`, `rowgen.go`), the API wiring in `internal/api/handlers.go`, and the shared math in `internal/finance/interest/`.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference under `internal/`.

Lineage. A faithful port of the DOS `Mortgage.pas`; for the original algorithm see the companion `legacy/code_docs/mortgage_dos_walkthrough.pdf`. This document explains the Go implementation.

Contents

0 How to read this document

1 Entry & data model

1.1 The pointer-field request · 1.2 The `MtgLine` row · 1.3 The continuous-rate boundary

2 The `Calc` dispatch

2.1 Pre-flight & balloon classification · 2.2 Summation · 2.3 The down-payment triad · 2.4 Price ↔ Monthly & balloon

3 Early termination & APR

4 APR crossover comparison

5 Row generation (what-if)

6 Result advisories

7 Math primitives & the monthly assumption

8 Appendix: call graph & file:line index

0 How to read this document

The Go mortgage engine sizes a loan behind a REST API. As with every `Per%Sense` screen, the caller leaves one field blank and the engine solves for it: leave the monthly payment blank to size a payment; leave the price blank to find the most expensive house a payment can buy; leave a balloon amount blank to size a balloon. Two comparison features sit on top: a single-loan APR (true yield once points are folded in) and a two-loan APR crossover.

Citation convention

References look like `mortgage.go:184` and resolve under `internal/finance/mortgage/` unless another path is named (e.g. `api/handlers.go:478`). Pascal lineage is noted as `Mortgage.pas:294`.

Two things to internalize. (1) Summation `mortgage.go:115` is the annuity factor at the heart of every formula. (2) The stored Rate is a continuously-compounded rate; the API converts to/from the displayed nominal-monthly yield at the boundary. The engine is monthly-only — the /12 is hard-coded throughout.

1 Entry & data model

1.1 The pointer-field request

The calc entry is `HandleMortgageCalc` `api/handlers.go:478`, decoding a `MortgageRequest` `api/handlers.go:55–72` whose optional inputs are pointers so an omitted key differs from a supplied zero. The handler maps each non-nil pointer to an `MtgLine` field with status `InOutInput`; a nil pointer leaves the status at `StatusEmpty` `api/handlers.go:490–537`. That presence/absence is the field-presence signal `Calc` reads.

```
Price *float64 `json:"price,omitempty"` // nil => blank, solve for it
Monthly *float64 `json:"monthly,omitempty"` // supply a value => InOutInput
Basis string `json:"basis,omitempty"` // APR day-count: "360" or default 365.25
```

1.2 The MtgLine row

The calculation row is `MtgLine` `mortgage.go:44–68` (float64 fields, distinct from any decimal storage type). Each value carries an int8 status:

Field	Meaning
Price / Points	purchase price / origination points (fraction)
Pct / Cash / Financed	the mutually-exclusive down-payment triad: down fraction / cash out of pocket / loan principal
Years / Rate	term (int) / the continuous true rate (see §1.3)
Tax / Monthly	flat monthly escrow (not amortized) / total monthly payment
When / HowMuch / BalloonStat	balloon timing in years (int) / amount / status enum (Known/Unk/Blank)

Note the mortgage row stores balloon timing as `When int` (years), not a date — the mortgage `Calc` path uses no `dateutil` date arithmetic.

1.3 The continuous-rate boundary

Gotcha. `MtgLine.Rate` is a continuous rate; the UI shows a nominal-monthly yield. Conversion happens only at the API boundary: inbound `LoanRateToTrueRate` `mortgage.go:159–162` (called at `api/handlers.go:521`), outbound `TrueRateToLoanRate` `mortgage.go:171–174` (called at `api/handlers.go:550`). Both wrap the interest package with a hard-coded $n=12$: `RateFromYield(y, 12) = 12 \cdot \ln(1+y/12)` and `YieldFromRate(r, 12) = 12 \cdot (e^{r/12} - 1)`. A port that stores the nominal yield in `Rate` corrupts every formula. The APR result is already a loan rate (the solver converts before returning), so the handler does not re-convert it `api/handlers.go:560–566`.

2 The Calc dispatch

`Calc(m)` `mortgage.go:184–309` takes `MtgLine` by value, mutates the copy, and dispatches purely on `*Status == InOutInput`. It mirrors the DOS dispatcher `Mortgage.pas:294` step for step (see the DOS companion §3).

2.1 Pre-flight & balloon classification

Before any solve, `Calc` runs the equivalent of the DOS `FirstPass` inline `mortgage.go:189–230`:

Check	Behavior	Line
Years input ≤ 0	hard error — term must be positive	<code>mortgage.go:189</code>
Rate $> 0.19835\dots$	non-blocking warning ($\approx 20\%$ nominal; usually a units slip)	<code>mortgage.go:205–212</code>
When+HowMuch both input	BalloonKnown — a fixed lump	<code>mortgage.go:215–217</code>
When only	BalloonUnk — solve for the amount	<code>mortgage.go:218</code>
HowMuch without When	hard error — must say when it is due	<code>mortgage.go:220–222</code>
Financed $>$ Price	non-blocking warning; still computes (negative %Down/Cash is the signal)	<code>mortgage.go:226–230</code>

The high-rate threshold `0.19835162342` `mortgage.go:37` is the true-rate form of 20% nominal, and it fires only on a user-entered rate, never a solved one — matching DOS `MortgageScreenUnit.pas:285`.

2.2 Summation — the annuity factor

`Summation(r, t)` `mortgage.go:115–132` is the present value (per unit monthly payment) of $12 \cdot t$ monthly payments at continuous rate r :

$$\text{Summation} = f \cdot (1 - \text{last}) / (1 - f), \quad f = e^{(-r/12)}, \quad \text{last} = e^{(-r \cdot t)} \\ \rightarrow 12 \cdot t \quad \text{as } r \rightarrow 0 \quad (\text{the teeny branch})$$

The teeny branch `mortgage.go:117` returns the undiscounted count $12 \cdot t$; a second guard catches a denominator collapse `mortgage.go:128`. The hard-coded `/12` is the monthly-payment assumption baked into the engine.

2.3 The down-payment triad

With `Price` input, `computeCashPctAndFinanced` `mortgage.go:314–342` derives the two of {Pct, Cash, Financed} that were not input (status `InOutOutput`):

$$\begin{aligned} \text{Cash} &= \text{Price} \cdot (\text{Pct} + (1 - \text{Pct}) \cdot \text{Points}) & \text{Financed} &= \text{Price} \cdot (1 - \text{Pct}) \\ \text{from Cash: } \text{Pct} &= (\text{Cash} / \text{Price} - \text{Points}) / (1 - \text{Points}) \\ \text{from Financed: } \text{Pct} &= 1 - \text{Financed} / \text{Price} \end{aligned}$$

A `Pct  $\geq 0.995$` result ($\approx 100\%$ down) is a hard error on both the Cash and Financed branches `mortgage.go:328 / 337` — there is no loan left to solve for.

2.4 Price ↔ Monthly & balloon

Once the triad, Years and Rate are present `mortgage.go:239`, the main block first values any known balloon ($\text{balloonval} = \text{HowMuch} \cdot e^{-(\text{Rate} \cdot \text{When})}$ `mortgage.go:242–249`), then dispatches on which of Price / Monthly is blank:

Known (input)	Solves	Formula	Line
Price + triad, Monthly blank	Monthly	$(\text{Price}(1-\text{Pct}) - \text{balloonval})/S(\text{Rate}, \text{Years}) + \text{Tax}$	<code>mortgage.go:278</code>
Monthly + Pct, Price blank	Price	$((\text{Monthly}-\text{Tax}) \cdot S + \text{balloonval})/(1-\text{Pct})$	<code>mortgage.go:291</code>
Monthly + Cash, Price blank	Price	$\text{Cash} + (1-\text{Points}) \cdot ((\text{Monthly}-\text{Tax}) \cdot S + \text{balloonval})$	<code>mortgage.go:293</code>
Price + Monthly, balloon Unk	HowMuch	$\text{bv} = \text{Price}(1-\text{Pct}) - (\text{Monthly}-\text{Tax}) \cdot S - \text{balloonval};$ $\text{HowMuch} = \text{bv} \cdot e^{(\text{Rate} \cdot \text{When})}$	<code>mortgage.go:346–357</code>

The flat Tax is added after amortization, never financed. Both Price and Monthly given with no unknown balloon is over-determined → error `mortgage.go:264`. After solving Price from Monthly, Calc back-fills the triad and flips PriceStatus to output `mortgage.go:299–303`.

DOS-faithful note. A balloonCalc solve deliberately leaves BalloonStat as BalloonUnk rather than promoting it to BalloonKnown, so a second CALC behaves identically — mirroring the DOS comment at `Mortgage.pas: 434` ("This messes things up if you CALC twice").

3 Early termination & APR

Two functions value a loan paid off early — the basis of the APR solver. `TerminalBalloon(ei, t)` `mortgage.go:367–387` is the outstanding balance t years out (including the payment due then):

$$\text{TB}(t) = [\text{Financed} - (\text{Monthly}-\text{Tax}) \cdot S(\text{Rate}, t-1/12) - [\text{paid?}] \cdot \text{HowMuch} \cdot e^{-(\text{Rate} \cdot \text{When})}] \cdot e^{(\text{Rate} \cdot t)}$$

`ValueOfPaymentsForTerminatedLoan(ei, r, t)` `mortgage.go:393–421` is the PV (at loan date, at an arbitrary trial rate r) of all cash if the loan terminates at t — discounting the terminal payoff at the trial rate is what lets the solver search for the yield.

`IterateToFindAPR` `mortgage.go:428–468` is a secant (finite-difference Newton) solve for the rate at which the discounted cash flows equal the net cash advanced:

```
target    = Financed · (1 - Points)          first guess apr = Rate + Points/Years
newdelta  = (target - value) · delta / (value - oldvalue)
report    = YieldFromRate(apr, 12)
```

≤20 iterations `mortgage.go:443`; converged when $|\text{delta}| < \text{tiny}$ `mortgage.go:462`. `FullTermAPR` `mortgage.go:472` uses $t = \text{Years} + 1/12$ (held to maturity); `OneMonthAPR` `mortgage.go:480` is the closed-form worst case (payoff at the first payment, with points fully amortized over one month). The handler calls `FullTermAPR` when `EnoughDataForAPR` holds `api/handlers.go:560–561`; non-convergence surfaces as a warning `api/handlers.go:570`.

4 APR crossover comparison

A crossover is the holding period at which two loans have equal effective APR. CompareAPRs

mortgage.go:505–584 first gates on each loan having enough data for an APR mortgage.go:511–517 (DOS Mortgage.pas:957–961), then runs the **dominance test**: a loan that wins at both the one-month and full-term horizons is "always better" — no crossover exists mortgage.go:531–540. Only when the ordering must flip *between* those horizons does the crossover solver run.

DOS-faithful note (the M6 fix). Two behaviors here were corrected to match the DOS engine exactly. (1) **Mismatched-term handling**: when the two loans have different Years and one dominates at both horizons, the result is a plain "Mortgage *N* is always better" mortgage.go:533 / 539 — the solver is *not* forced to manufacture a crossover. The crossover branch only runs when the dominance test fails. (2) **Grammar**: the duration string emits DOS's plural forms — "%d years" and "%d months" mortgage.go:560–572 — so "1 years, 1 months" is intentional, matching Mortgage.pas:1001–1006. Do not singularize.

iterateToFindCrossoverAPRandTime mortgage.go:587–725 solves simultaneously for the rate *r* and time *t* at which both loans price to par. The per-loan residual is $1 - V(r, t) / (\text{Financed} \cdot (1 - \text{Points}))$ mortgage.go:591; a 2×2 Jacobian is built by finite differences and a Newton step taken via the inverse Jacobian (Cramer's rule):

$$\begin{aligned} \det &= \partial T_1 / \partial t \cdot \partial T_2 / \partial r - \partial T_1 / \partial r \cdot \partial T_2 / \partial t \\ dr &= (\partial T_2 / \partial t \cdot T_1 - \partial T_1 / \partial t \cdot T_2) / \det \\ dt &= (-\partial T_2 / \partial r \cdot T_1 + \partial T_1 / \partial r \cdot T_2) / \det \end{aligned}$$

First guess $t = 0.25 \cdot (\text{Years}_1 + \text{Years}_2)$, rate seeded from the mean full-term APR (or the rate-plus-points mean if the APRs did not converge) mortgage.go:599–613; ≤40 iterations to both residuals < teeny; report YieldFromRate(*r*, 12) with *t* floored to the previous whole month mortgage.go:715–719.

Balloon-date discontinuity fallback. The terminated-loan value is discontinuous at a balloon date, so the Newton iteration can stall. tryBalloonDates mortgage.go:728–760 samples each balloon-bearing loan's APR just before and just after the date; if the APR ordering between the two loans flips across it, the crossover is pinned to that balloon date mortgage.go:702–714. A port that omits this fails to converge on balloon loans.

5 Row generation (what-if)

What-if tables vary one input across rows and re-solve the dependent field. EnoughDataForRowGeneration

rowgen.go:38–42 requires the base row to have computed a Price, Monthly, or HowMuch (status InOutOutput) — that output field is the one re-solved per row (DOS Mortgage.pas:1075–1078).

GenerateRows(base, vary, inc, n) rowgen.go:58–109 walks *n* rows: after the first, it bumps the varied field by *inc* (bumpField), then blanks whichever of {Price, Monthly, HowMuch} was the base output and re-runs Calc rowgen.go:80–104. bumpField rowgen.go:111–149 applies a rate increment in loan-rate (yield) space, not continuous space rowgen.go:124, so "+0.5%" means 0.5% of nominal rate — matching DOS CopyAndIncrement Mortgage.pas:1228–1233, which converts the rate column to a yield, adds the step, and converts back.

GenerateGrid rowgen.go:156 sweeps a second axis by stepping the secondary field once per band and delegating each band to GenerateRows. Both axes are bounded by MaxWhatIfRows = 1000 rowgen.go:56.

6 Result advisories

After a successful solve, `appendResultAdvisories` `advisories.go:16` runs a sanity pass over the solved cells and appends non-blocking advisories to `result.Warnings`. It never changes a computed number. The codes (M-W1...M-W7) cover balloon degeneracies `advisories.go:38–62` (essentially-zero, negative, or principal-exceeding balloons), negative %Down when financed exceeds price `advisories.go:69`, non-positive solved Price or negative Cash `advisories.go:76–82`, and a terminal balloon at the final year `advisories.go:90`. M-W8 (non-converged APR) is emitted by the handler instead, where the APR is computed.

7 Math primitives & the monthly assumption

The shared interest package: `Exxp/Lnn` `interest/math.go:43 / 67` (safe exp/ln with Taylor guards near the origin — the `Lnn` guard compensates for a Turbo Pascal precision bug for x near 1), and `RealPerYr/YieldFromRate/RateFromYield` `interest/rates.go:42 / 63 / 79` ($nn \cdot (e^{r/nn} - 1)$ and $nn \cdot \ln(1 + y/nn)$).

The engine is monthly-only. Every mortgage call passes $n=12$: Summation uses $f = e^{-r/12}$ and the zero-rate fallback 12·t; the boundary converters and APR reporters all use 12. So `RealPerYr` returns 12 throughout and `YieldFromRate/RateFromYield` reduce to the nominal-monthly $\leftrightarrow$ continuous conversions. The `ydays` argument (360 vs 365.25) flows in only via `mtgAPRYrDays(Basis)` `api/handlers.go:76` for the APR day-count and is otherwise inert.

8 Appendix: call graph & file:line index

POST /api/mortgage/calc -> HandleMortgageCalc	api/handlers.go:478
- build MtgLine (pointer fields -> status)	api/handlers.go:490
- LoanRateToTrueRate (loan rate -> continuous)	api/handlers.go:521 / mortgage.go:159
- mortgage.Calc(m)	mortgage.go:184
- pre-flight + balloon classification	mortgage.go:189
- computeCashPctAndFinanced (Pct<->Cash<->Financed)	mortgage.go:314
- Summation $f \cdot (1 - \text{last}) / (1 - f)$	mortgage.go:115
- solve Monthly Price balloonCalc	mortgage.go:278 / 291 / 346
\ - appendResultAdvisories (M-W1..M-W7)	advisories.go:16
- EnoughDataForAPR -> FullTermAPR	api/handlers.go:560 / mortgage.go:472
\ - IterateToFindAPR (secant)	mortgage.go:428
\ - ValueOfPaymentsForTerminatedLoan -> TerminalBalloon	mortgage.go:393 / 367
\ - TrueRateToLoanRate (continuous -> loan rate)	api/handlers.go:550 / mortgage.go:171
POST /api/mortgage/compare -> HandleMortgageCompare	api/handlers.go:625
\ - CompareAPRs	mortgage.go:505
- FullTermAPR x2, OneMonthAPR x2 (dominance)	mortgage.go:519 / 480
\ - iterateToFindCrossoverAPRAndTime (2-var Newton)	mortgage.go:587
\ - tryBalloonDates (discontinuity fallback)	mortgage.go:728
POST /api/mortgage/whatif -> HandleMortgageWhatIf	api/handlers.go:696
\ - GenerateRows / GenerateGrid	rowgen.go:58 / 156

Symbol	Reference
HandleMortgageCalc / boundary converters	<code>api/handlers.go:478</code> · <code>mortgage.go:159 / 171</code>
Calc / computeCashPctAndFinanced / balloonCalc	<code>mortgage.go:184 / 314 / 346</code>
Summation	<code>mortgage.go:115</code>
CompareAPRs / crossover / tryBalloonDates	<code>mortgage.go:505 / 587 / 728</code>
GenerateRows / bumpField / EnoughDataForRowGeneration	<code>rowgen.go:58 / 111 / 38</code>
appendResultAdvisories	<code>advisories.go:16</code>